Milestone 5 – CacheManager+BST, Due December 2, 2026 4:00 PM Pacific

The following files will be provided to you, for completion of your milestone:

- binary_search_tree.h // header file containing binary search tree class
- cache_manager.h // header file containing cache manager class
- dll_node.cpp // dll node file constructor
- dll_node.h // header file containing doubly linked node structure
- doubly_linked_list.h // header file containing doubly linked list class
- json.hpp // header file for processing json files
- hash_node.cpp // hash node file constructor
- hash_node.h // header file containing hash node structure
- hash_table.h // header file containing hash table class
- milestone5.h // header file containing log function
- milestone5.json // json file containing test cases and its transactions
- milestone5_config.json // json configuration (properties) file
- outputFile – testcase 1.txt // expected output for testcase 1
- tree_node.cpp // tree node file constructor
- tree_node.h // header file containing tree node structure
- milestone5.cpp /* cpp file containing main, which does the following:
    - Reads configuration file (json format) to:
        - retrieve inputFile (test case file (json format)
        - retrieve outputFile (text file containing generated output)
        - retrieve errorLogFile (text file containing error messages)
    - process inputFile test cases
    - write output to outputFile */

1. Write a Unified Modeling Language (UML) Class Diagram and Sequence Diagram for this milestone.
2. Write a FIFO list and basic hash table implementation, which uses the files listed above, and includes the following in separate cpp files:

    - cache_manager.cpp – implementation file that contains the following methods:

        1. getTable - return the hash table
        2. getList - return the FIFO list
        3. getBst – return the binary search tree
        4. getSize - return the number of items in the CacheManager
        5. isEmpty - Check if the HashTable is empty
        6. add - adds a new node to the CacheManager
        7. remove – remove node with key value

8. clear - remove all entries from the CacheManager
9. getItem - retrieve item from the CacheManager
10. getMaxCacheSize - retrieve max size of cache from the CacheManager
11. contains - check if a node with key exists in the table
12. printCache – prints out the contents of the cache
13. printRange – prints out the contents with keys from low to high
14. printSort – prints out the contents in either ascending or descending order

- doubly_linked_list.cpp – implementation file that contains the following methods:

15. getSize - Get the size of the list
16. isEmpty - Check if the list is empty
17. insertAtHead - Adds a new node at the beginning of the list
18. insertAtTail - Adds a new node at the end of the list
19. remove - Searches for a node with a specific value and deletes it from the list
20. removeHeaderNode – Removes header node
21. removeTailNode – Removes tail node
22. moveNodeToHead – Moves a specific node to the front
23. moveNodeToTail – Moves a specific node to the end
24. clear - Clear the list (delete all nodes)
25. printList – print the doubly linked list from head to tail to console and output file
26. reversePrintList - print the doubly linked list from tail to head to console and output file

- hash_table.cpp – implementation file that contains the following methods:

27. getTable - return the hash table
28. getSize - return the size of the hash table
29. calculateHashCode – perform hashing function
30. isEmpty - Check if the HashTable is empty
31. getNumberOfItems - return number of items in the hash table
32. add - adds a new node to the hash table
33. remove – remove node with key value
34. clear - remove all entries from the table
35. getItem - returns pointer to the HashNode
36. contains - check if a node with key exists in the table
37. printTable - print out the contents of table to console and output file

- binary_search_tree.cpp – implementation file that contains the following methods:

38. addToTree – add a FIFO node to the tree

39. removenode – remove a node from the tree
40. getHeightOfTree – calculate the height of the tree
41. getNumberOfTreeNodes – number of nodes in the tree
42. contains – true if the tree contains a key, false otherwise
43. getRoot – return the root node of the tree
44. isEmpty – returns true if the tree is empty, false otherwise
45. clear – clears out all of the nodes in the tree
46. printNodeFromTree – print out the data from a node to console and to file
47. printInOrder – traverse the tree in-order and print the key to console and to file
48. printReverseOrder – traverse the tree in reverse order and print the key to console and to file
49. printPreOrder – traverse the tree in pre-order and print the key to console and to file
50. printPostOrder – traverse the tree in post-order and print the key to console and to file
51. printDepthFirst – traverse the tree in depth first order and print the key to console and to file
52. printBreadthOrder – traverse the tree in breadth order and print the key to console and to file
53. printRange – print out the range of keys between low and high params to console and to file
54. deleteTree - deletes the tree starting from the specified node
55. getHeight - helper function to calculate the height of a node
56. printInOrderHelper - helper function for recursive in-order traversal
57. printReverseOrderHelper - helper function for recursive reverse in-order traversal
58. printPreOrderHelper - helper function for recursive pre-order traversal
59. printPostOrderHelper - helper function for recursive post-order traversal
60. printRangeHelper - helper function to traverse and print out the cache information given a low and high value

The total number of points for this milestone is 170, which will be based upon the following:

- All file(s) are submitted via Canvas (5)
- Each submitted/modified file must have student's name (-10% of total milestone points if missing)
- The UML Class Diagram and Sequence Diagram are documented (20)
- Each submitted/modified file must include description of changes made to a program, and its change date (4)
- Program compiles with all of the provided files (1)
- The following methods are commented:

    1. getTable - return the hash table (0.5)

2. getList - return the FIFO list (0.5)
3. getBst – return the binary search tree (0.5)
4. getSize - return the number of items in the CacheManager (0.5)
5. isEmpty - Check if the HashTable is empty (0.5)
6. add - adds a new node to the CacheManager (0.5)
7. remove – remove node with key value (0.5)
8. clear - remove all entries from the CacheManager (0.5)
9. getItem - retrieve item from the CacheManager (0.5)
10. getMaxCacheSize - retrieve max size of cache from the CacheManager (0.5)
11. contains - check if a node with key exists in the table (0.5)
12. printCache – prints out the contents of the cache (0.5)
13. printRange – prints out the contents with keys from low to high (0.5)
14. printSort – prints out the contents in either ascending or descending order (0.5)
15. getSize - Get the size of the list (0.5)
16. isEmpty - Check if the list is empty (0.5)
17. insertAtHead - Adds a new node at the beginning of the list (0.5)
18. insertAtTail - Adds a new node at the end of the list (0.5)
19. remove - Searches for a node with a specific value and deletes it from the list (0.5)
20. removeHeaderNode – Removes header node (0.5)
21. removeTailNode – Removes tail node (0.5)
22. moveNodeToHead – Moves a specific node to the front (0.5)
23. moveNodeToTail – Moves a specific node to the end (0.5)
24. clear - Clear the list (delete all nodes) (0.5)
25. printList – print the doubly linked list from head to tail to console and output file (0.5)
26. reversePrintList - print the doubly linked list from tail to head to console and output file (0.5)
27. getTable - return the hash table (0.5)
28. getSize - return the size of the hash table (0.5)
29. calculateHashCode – perform hashing function (0.5)
30. isEmpty - Check if the HashTable is empty (0.5)
31. getNumberOfItems - return number of items in the hash table (0.5)
32. add - adds a new node to the hash table (0.5)
33. remove – remove node with key value (0.5)
34. clear - remove all entries from the table (0.5)
35. getItem - returns pointer to the HashNode (0.5)
36. contains - check if a node with key exists in the table (0.5)
37. printTable - print out the contents of table to console and output file (0.5)
38. addToTree – add a FIFO node to the tree (0.5)
39. removenode – remove a node from the tree (0.5)
40. getHeightOfTree – calculate the height of the tree (0.5)
41. getNumberOfTreeNodes – number of nodes in the tree (0.5)
42. contains – true if the tree contains a key, false otherwise (0.5)

43. getRoot – return the root node of the tree (0.5)
44. isEmpty – returns true if the tree is empty, false otherwise (0.5)
45. clear – clears out all of the nodes in the tree (0.5)
46. printNodeFromTree – print out the data from a node to console and to file (0.5)
47. printInOrder – traverse the tree in-order and print the key to console and to file (0.5)
48. printReverseOrder – traverse the tree in reverse order and print the key to console and to file (0.5)
49. printPreOrder – traverse the tree in pre-order and print the key to console and to file (0.5)
50. printPostOrder – traverse the tree in post-order and print the key to console and to file (0.5)
51. printDepthFirst – traverse the tree in depth first order and print the key to console and to file (0.5)
52. printBreadthOrder – traverse the tree in breadth order and print the key to console and to file (0.5)
53. printRange – print out the range of keys between low and high params to console and to file (0.5)
54. deleteTree - deletes the tree starting from the specified node (0.5)
55. getHeight - helper function to calculate the height of a node (0.5)
56. printInOrderHelper - helper function for recursive in-order traversal (0.5)
57. printReverseOrderHelper - helper function for recursive reverse in-order traversal (0.5)
58. printPreOrderHelper - helper function for recursive pre-order traversal (0.5)
59. printPostOrderHelper - helper function for recursive post-order traversal (0.5)
60. printRangeHelper - helper function to traverse and print out the cache information given a low and high value (0.5)

- The following methods run without errors:

1. getTable - return the hash table (1)
2. getList - return the FIFO list (1)
3. getBst – return the binary search tree (1)
4. getSize - return the number of items in the CacheManager (1)
5. isEmpty - Check if the HashTable is empty (1)
6. add - adds a new node to the CacheManager (1)
7. remove – remove node with key value (1)
8. clear - remove all entries from the CacheManager (1)
9. getItem - retrieve item from the CacheManager (1)
10. getMaxCacheSize - retrieve max size of cache from the CacheManager (1)
11. contains - check if a node with key exists in the table (1)
12. printCache – prints out the contents of the cache (1)
13. printRange – prints out the contents with keys from low to high (1)
14. printSort – prints out the contents in either ascending or descending order (1)

15. getSize - Get the size of the list (1)
16. isEmpty - Check if the list is empty (1)
17. insertAtHead - Adds a new node at the beginning of the list (1)
18. insertAtTail - Adds a new node at the end of the list (1)
19. remove - Searches for a node with a specific value and deletes it from the list (1)
20. removeHeaderNode – Removes header node (1)
21. removeTailNode – Removes tail node (1)
22. moveNodeToHead – Moves a specific node to the front (1)
23. moveNodeToTail – Moves a specific node to the end (1)
24. clear - Clear the list (delete all nodes) (1)
25. printList – print the doubly linked list from head to tail to console and output file (1)
26. reversePrintList - print the doubly linked list from tail to head to console and output file (1)
27. getTable - return the hash table (1)
28. getSize - return the size of the hash table (1)
29. calculateHashCode – perform hashing function (1)
30. isEmpty - Check if the HashTable is empty (1)
31. getNumberOfItems - return number of items in the hash table (1)
32. add - adds a new node to the hash table (1)
33. remove – remove node with key value (1)
34. clear - remove all entries from the table (1)
35. getItem - returns pointer to the HashNode (1)
36. contains - check if a node with key exists in the table (1)
37. printTable - print out the contents of table to console and output file (1)
38. addToTree – add a FIFO node to the tree (1)
39. removenode – remove a node from the tree (1)
40. getHeightOfTree – calculate the height of the tree (1)
41. getNumberOfTreeNodes – number of nodes in the tree (1)
42. contains – true if the tree contains a key, false otherwise (1)
43. getRoot – return the root node of the tree (1)
44. isEmpty – returns true if the tree is empty, false otherwise (1)
45. clear – clears out all of the nodes in the tree (1)
46. printNodeFromTree – print out the data from a node to console and to file (1)
47. printInOrder – traverse the tree in-order and print the key to console and to file (1)
48. printReverseOrder – traverse the tree in reverse order and print the key to console and to file (1)
49. printPreOrder – traverse the tree in pre-order and print the key to console and to file (1)
50. printPostOrder – traverse the tree in post-order and print the key to console and to file (1)
51. printDepthFirst – traverse the tree in depth first order and print the key to console and to file (1)

52. printBreadthOrder – traverse the tree in breadth order and print the key to console and to file (1)
53. printRange – print out the range of keys between low and high params to console and to file (1)
54. deleteTree - deletes the tree starting from the specified node (1)
55. getHeight - helper function to calculate the height of a node (1)
56. printInOrderHelper - helper function for recursive in-order traversal (1)
57. printReverseOrderHelper - helper function for recursive reverse in-order traversal (1)
58. printPreOrderHelper - helper function for recursive pre-order traversal (1)
59. printPostOrderHelper - helper function for recursive post-order traversal (1)
60. printRangeHelper - helper function to traverse and print out the cache information given a low and high value (1)

- The following test cases are processed, and produce expected output (10 per test case; 50 total)
- Extra Credit – use industry standard test program and/or extract test cases, in separate json test file